

Charles IACOPINO
Ju-Heon HWANG
Shubham RAJ BHATTARAI

Everest

Design brief.....	4
Delivery time.....	5
Working.....	5
Recipe.....	5
Documentation.....	6
DATA AND FILE HANDLING.....	6
FLASK APPLICATION.....	7
ROUTES DOCUMENTATION.....	7
API Routes (/api).....	7
GET /api/ping.....	7
POST /api/echo.....	7
Web Routes (/home).....	8
GET /home/hello/<name>.....	8
Authentication Routes.....	8
GET /login/.....	8
POST /login/.....	8
GET /register/.....	8
POST /register/.....	9
GET /logout/.....	9
Upload Routes (/upload).....	9
GET /upload/.....	9
POST /upload/submit.....	9
Route Protection.....	10
CLI TOOLS.....	10
ocr.py.....	10
export_md.py.....	10
format_small_book.py.....	10
moderate.py.....	10
log.py.....	10
Budget.....	10
Maintenance.....	11
Technologies.....	12
Web Interface: HTML, Flask Templates, and Bootstrap.....	12
Backend Language and Environment: Python and Poetry.....	12
Key Python Dependencies and Their Roles.....	13
Communication.....	13
Traceability.....	14
Progression.....	15
Phase 1 – Analysis and Conceptual Foundations.....	15
Phase 2 – UML Modeling and Scenario Definition.....	15

Phase 3 – Architectural Choices and Modular Design.....	16
Phase 4 – Incremental Implementation and MVP Validation.....	17
Phase 5 – Peer Review, Stabilization, and Documentation.....	17
Phase 6 – Work left.....	17
Scenario list.....	18
1. Login (connexion).....	19
2. User Registration (inscription).....	19
3. Permissions Management (permissions).....	19
4. Duplicate Upload ().....	20
5. Poor Quality Image Upload (upload_images_mauvaise_qualite).....	20
6. Incomplete Work.....	20
7. Invalid Structure ().....	21
8. Incorrect Book Identification.....	21
9. Upload of Inappropriate / Copyrighted Content.....	22
10. Abusive Behavior.....	22
11. Admin Management (gestion_admin).....	22
12. Server Overload (serveur_surchage).....	23
Use case.....	23
Unit tests results.....	24
Moderate.....	24
FS tests.....	24
Integration tests results.....	24
Validation tests results.....	25
Reversibility.....	25

Intro/context

This report presents the design and implementation of **Biblioteko**, a group university project developed over the course of a semester. The objective of the project was to explore the technical feasibility of a Decentralized Digital Library capable of processing scanned books using modern OCR techniques and Large Language Models (LLMs), while respecting legal constraints related to copyrighted content.

The project was deliberately developed as a **Minimum Viable Product (MVP)**. Rather than aiming for feature completeness or industrial deployment, the focus was placed on validating core functionalities, automation workflows, and architectural choices within limited time constraints. Particular attention was given to document processing pipelines, content structuring, and the prevention of illegal storage of protected works.

Biblioteko provides a web interface and command-line tools that allow users to upload scanned documents, extract structured text through OCR and LLM-based post-processing, and generate clean Markdown representations of book content. The system also integrates moderation mechanisms to identify and reject copyrighted material before storage.

This document details the project from both a **client-oriented perspective** (design brief, delivery constraints, documentation, maintenance considerations) and a **developer-oriented perspective** (architecture, scenarios, traceability, testing, and reversibility). It aims to clearly describe what was implemented, how it works, and which design decisions were made, while explicitly acknowledging the limitations inherent to an MVP approach.

Client

Design brief

The objective of the project is to design a Minimum Viable Product (MVP) for a dematerialized library system capable of:

1. Accepting scanned book documents,
2. Performing OCR and structured text extraction using Large Language Models,
3. Producing a clean Markdown representation of the extracted content,
4. Preventing the storage of copyrighted works.

The project focuses on technical feasibility, legal constraints, and automation rather than completeness or industrial deployment.

Delivery time

The project was developed over the course of a university semester. The effective implementation phase was short and focused, aligning with the MVP scope and prioritizing functional validation over feature completeness.

Working

The project followed an iterative working method inspired by **KANBAN**.

This method was used to:

1. Brainstorm features,
2. Prioritize tasks,
3. Identify risks and edge cases,
4. Decide which elements belonged to the MVP scope.

The KANBAN method was used to track ideas, scenarios, and implementation status, enabling flexible adaptation throughout the project.

Recipe

This section describes the operational workflow of the Biblioteko MVP, from document submission to final output generation.

1. User Authentication

The user accesses the application through the web interface and authenticates using valid credentials. Authentication is required to access all protected features, including uploads and document processing.

2. Document Upload

Once authenticated, the user uploads a scanned document (PDF or image) via the upload interface. The file is temporarily stored on the server for processing.

3. Image Preprocessing

Uploaded documents may be preprocessed using CLI tools to improve OCR quality. This step includes image rotation, cropping, contrast enhancement, and noise reduction.

4. **OCR Extraction**

The processed images or PDF pages are sent to a Large Language Model acting as an OCR engine. Raw text and metadata (page number, language, text quality) are extracted and stored.

5. **Content Modération**

Before permanent storage, the system evaluates the document's metadata to detect potentially copyrighted content. If the document is classified as copyrighted, processing is stopped and the content is rejected.

6. **Markdown Formatting**

Valid OCR output is passed to a formatting module that converts raw text into clean, structured Markdown. The formatter enforces consistent structure, page markers, and metadata headers.

7. **Storage and Access**

The resulting Markdown files are stored in a structured directory hierarchy. The user can browse processed books and pages through the web interface.

Documentation

Can also be found in the file : [src/readme.md](#)

Code has no magic numbers, comments and self-explanatory naming.

DATA AND FILE HANDLING

Uploads Raw Files Folder

uploads/raw_files

This directory temporarily stores raw uploaded files such as:

- PDF files
- Images
- Archives

These files are processed and later converted into structured data.

Scans Folder

data/sequestre/scans

This directory stores processed book scans.

Structure:

data/sequestre/scans / book_name / book_name_p<number>

One folder per book

One file per page

Used during OCR and post-processing

FLASK APPLICATION

Application Entry Point

The Flask application starts from:

src/[app.py](#)

Key responsibilities:

- Application creation using the factory pattern
- Blueprint registration
- Session-based authentication
- Global authentication protection using a before-request hook

Unauthenticated users are redirected to the login page.

ROUTES DOCUMENTATION

This section lists all available routes exposed by the application, including web routes and API endpoints.

API Routes (/api)

These routes expose JSON endpoints intended for programmatic access and health checks.

GET /api/ping

Description:

Health check endpoint used to verify that the API is reachable.

Response:

Returns a simple JSON message confirming the API is running.

POST /api/echo

Description:

Debug endpoint that echoes back the JSON payload sent in the request.

Request body:

JSON object (any structure)

Response:

Returns the received JSON payload.

Web Routes (/home)

These routes render HTML templates and are intended for end users.

GET /home/hello/<name>**Description:**

Test route that displays a greeting message using a URL parameter.

Parameters:

1. **name:** string

Response:

Returns a simple greeting message.

Authentication Routes

These routes manage user authentication and session handling.

GET /login/**Description:**

Displays the login form.

POST /login/**Description:**

Authenticates a user using provided credentials.

Effect:

1. Creates a user session on success
2. Redirects to the homepage

GET /register/**Description:**

Displays the user registration form.

POST /register/

Description:

Creates a new user account.

Effect:

1. Validates input data
2. Stores the user securely
3. Returns a JSON confirmation

GET /logout/

Description:

Logs out the current user.

Effect:

1. Clears the session
2. Redirects to the homepage

Upload Routes (/upload)

These routes handle file uploads through the web interface.

GET /upload/

Description:

Displays the file upload form.

POST /upload/submit

Description:

Handles file submission from the upload form.

Effect:

1. Validates uploaded file
2. Saves file to the server
3. Displays success or error message

Route Protection

1. All routes are **protected by default**
2. Unauthenticated users are redirected to the login page
3. Exceptions:
 - a. Login
 - b. Register
 - c. Static assets
 - d. API health routes (e.g. [/api/ping](#))

CLI TOOLS

The cli directory contains command-line utilities for offline processing.

ocr.py

Sends scanned book pages to a Large Language Model and extracts raw text and metadata.

export_md.py

Converts OCR results into structured Markdown using an LLM.

format_small_book.py

Image preprocessing tool used to crop and zoom pages to improve OCR quality.

moderate.py

Determines whether a book is likely copyrighted.

log.py

Centralized logging utility for CLI scripts.

Budget

As a university project developed as a Minimum Viable Product (MVP), Biblioteko does not rely on a production-grade infrastructure and therefore incurs minimal operational costs.

The primary cost factors identified are:

- 1) Development time, provided by the project team as part of academic work
- 2) External API usage, mainly Large Language Model (LLM) services used for OCR extraction, text formatting, and content moderation
- 3) Hosting resources, limited to a lightweight server for running the Flask application and storing temporary files
- 4) No dedicated database, cloud storage service, or paid frontend framework is required in the current implementation.

As a result, the overall budget remains very low and suitable for experimentation and prototyping. In a production context, costs would mainly scale with document volume and LLM usage, and could be controlled through batching, caching, or alternative OCR solutions.

Maintenance

The application was designed to be easy to understand and maintain, even by developers who were not part of the original team.

Maintenance starts with the project documentation, primarily available in the [src/readme.md](#) file, which explains the project structure, setup steps, and execution procedures. Dependency management is handled using Poetry, ensuring a reproducible and consistent development environment.

The architecture follows a modular organization:

1. Web routes and authentication logic are separated into Flask blueprints
2. OCR, formatting, and moderation are implemented as independent CLI tools
3. File storage follows a predictable directory structure

This separation allows a maintainer to quickly identify where a given feature is implemented and modify it without impacting unrelated components.

To maintain or extend the application, a developer can:

1. Set up the environment using Poetry

2. Run the Flask application locally
3. Test individual processing steps using the CLI tools
4. Replace or update external services (e.g. LLM providers) without modifying the core workflow

The MVP avoids complex infrastructure (databases, message queues, distributed services), which significantly reduces maintenance overhead. This makes the project suitable for learning, experimentation, and gradual evolution.

Technologies

Web Interface: HTML, Flask Templates, and Bootstrap

The web interface of the MVP is built using **HTML** rendered through **Flask templates**, with **Bootstrap** for styling and layout. This stack was chosen primarily for its simplicity and speed of implementation.

Flask templates allow rapid server-side rendering without introducing unnecessary frontend complexity (such as a full SPA framework), which is well suited for an MVP whose goal is to validate functionality rather than UI sophistication. Bootstrap provides a ready-to-use, responsive design system that significantly reduces development time while ensuring a clean and usable interface across devices.

Overall, this approach enables fast iteration, low maintenance overhead, and clear separation between backend logic and presentation, which is sufficient and appropriate for a proof-of-concept.

Backend Language and Environment: Python and Poetry

The backend is implemented in **Python 3.11**, with dependency management handled by **Poetry**.

Python was selected because it is a **high-level, third-generation language** that is close to pure algorithmic thinking and widely used as a “glue language” to connect heterogeneous systems. Its expressive syntax, large standard library, and strong ecosystem make it particularly effective for rapidly prototyping data-processing pipelines, API integrations, and AI-driven workflows.

From an MVP perspective, Python allows the team to focus on *what* the system does rather than *how* low-level mechanisms are implemented. Additionally, the abstraction level of

Python makes the core business logic relatively easy to port to another language or framework in the future if scalability or performance requirements evolve.

Poetry was chosen to manage dependencies and virtual environments in a clean and reproducible way. It simplifies project setup, ensures consistent dependency resolution, and aligns with modern Python development best practices.

Key Python Dependencies and Their Roles

The project relies on the following categories of libraries:

1. **Web framework**
 - **flask**: lightweight and flexible backend framework, ideal for MVPs.
 - **python-dotenv**: environment variable management for configuration and API keys.
2. **Document and image processing**
 - **pillow**: image manipulation and preprocessing for scanned pages.
 - **pymupdf / fitz**: PDF parsing and page extraction, enabling efficient handling of scanned documents.
3. **AI and LLM integration**
 - **google-generativeai**
 - **mistralai**
4. These libraries provide access to Large Language Models used for OCR post-processing, text structuring, and Markdown formatting. Using external LLM APIs avoids building and maintaining complex OCR and NLP pipelines internally, which is consistent with the MVP philosophy.
5. **Networking and utilities**
 - **requests**: simple and reliable HTTP client for API calls.
 - **google**: auxiliary tooling for Google ecosystem integration.
6. **Security**
 - **argon2-cffi**: secure password hashing algorithm, ensuring basic authentication safety even at the MVP stage. Much better than md5 (security wise) or bcrypt (performance wise).

Communication

Communication with the client was considered an integral part of the project, despite the academic context and the absence of a commercial stakeholder. The “client” role was represented by the project brief and the supervising teaching staff, acting as a proxy for a real-world stakeholder.

From the beginning of the project, the objectives and limitations of Biblioteko were clearly communicated. In particular, it was explicitly stated that the goal was not to deliver a production-ready system, but a Minimum Viable Product (MVP) intended to validate technical feasibility, architectural choices, and legal constraints related to digital libraries.

Regular exchanges focused on:

- clarifying functional expectations,
- justifying technical and architectural decisions (diagrams) ,
- discussing legal and ethical constraints related to copyrighted content.

Key design decisions, such as the use of Large Language Models for OCR post-processing and text structuring, the absence of a traditional database, and the separation between the web application and command-line processing tools, were documented and explained to ensure transparency and shared understanding.

Communication relied on several supports:

- project documentation (README and technical descriptions)

This approach ensured that the client remained informed of the project's state, understood the trade-offs made during development, and validated the core functionalities progressively.

Developers

Traceability

Project traceability was ensured through:

1. Git version control for code evolution, with a dedicated *dev* **branch** used as the main integration branch. From this *dev* **branch**, each team member created their own feature branch to work independently, ensuring a clear separation of concerns, safer development, and easier code reviews before merging.
2. KANBAN boards to track ideas, scenarios, and implementation decisions,
3. Scenario files documenting expected system behaviors.

Progression

The progression of the Biblioteko project strictly followed the structure and pedagogical objectives defined in the supervised course. The project was conceived as a Minimum Viable Product (MVP) .I

Work was organized around heterogeneous pairs, each responsible for a specific subset of the system. This organization encouraged autonomy, accountability, and knowledge sharing, while reflecting realistic software engineering practices.

Phase 1 – Analysis and Conceptual Foundations

The project began with an extensive **analysis phase**, as required by the course guidelines.

The primary objective of this phase was to establish a shared and precise understanding of the problem domain.

Key activities included:

- identification of core concepts (entities, roles, actions, properties) from the project brief,
- creation of a **business glossary** and a **technical glossary**, with explicit justification of naming choices,
- alignment of vocabulary with object-oriented design principles and the importance of naming, as emphasized in the course material.

This phase provided the semantic foundation necessary to ensure consistency between requirements, models, code, and documentation.

Phase 2 – UML Modeling and Scenario Definition

Following the analysis, the project entered a **model-driven design phase**, strongly inspired by the AUP methodology and the Y-shaped development cycle.

During this phase:

- a global **use case diagram** was produced to capture the main interactions between users and the system,
- fundamental scenarios were identified, refined, and prioritized (e.g. proposing a document, OCR processing, moderation, Markdown export),
- several **system sequence diagrams** were created to describe the dynamic behavior of the most critical scenarios,
- **class diagrams** were designed to model the structure of the system and its key components,
- **activity diagrams** were used to represent document workflows, especially moderation and processing pipelines.

Modeling artifacts were produced using textual diagram tools (Mermaid), ensuring traceability and versioning within the Git repository.

Phase 3 – Architectural Choices and Modular Design

Based on the previous modeling work, architectural decisions were progressively refined.

The project adopted a **modular and lightweight architecture**, consistent with MVP constraints and the course’s emphasis on maintainability and evolvability.

Key architectural principles included:

- separation of concerns between the web interface and document processing logic,
- use of command-line tools for OCR, formatting, and moderation, enabling independent testing and reuse,
- file-based storage organized as a directory hierarchy, aligned with the “decentralized library” metaphor,
- avoidance of unnecessary infrastructure (databases, distributed services) to reduce complexity.

Design choices were explicitly justified with respect to **quality attributes** such as modularity, clarity, testability, and maintainability, as required by the course.

Phase 4 – Incremental Implementation and MVP Validation

Implementation was carried out in **short, incremental iterations**. Each iteration focused on validating a concrete capability of the system rather than implementing all planned features.

The implemented MVP includes:

- user authentication and access control,
- document upload and temporary storage,
- OCR extraction using Large Language Models,
- Markdown generation and structured export.

At each step, functionality was validated either through the web interface or via command-line execution, ensuring that modules remained independent and testable.

Phase 5 – Peer Review, Stabilization, and Documentation

Throughout the project, **peer evaluation** played an important role.

Pairs regularly reviewed the work of other teams, providing feedback on:

- architectural coherence,
- clarity of models and documentation,
- relevance of design decisions.

This cross-evaluation process helped improve both the quality of the project and the team's own practices.

Phase 6 – Work left

User case :

We can probably add the functions of removing a user. Also to administrate the users for the admin user.

Upload :

The moderation module is still synchronous. It's easier to send a response to the user but locks the user in the upload form while the file is being analyzed by the LLMs (and at some point the APIs more on that in the next point). Changing it to asynchronous would allow the user to do other things

but the server would need to implement a notification system to notify the user that their file has been uploaded and properly scanned (if the moderation module didn't recognize it as a copyrighted work).

Moderation :

The APIs are not implemented. With the [copyrighted book](#) provided as an example in the git repo. We also found another [French API](#) but did not look into it that much.

Scenario list

Can be found in [docs/fr/Conception/Scenarios](#) :

- comportement_abusif
- connexion
- connexion_suspecte
- gestion_admin
- inscription
- mauvais_livre_identifie
- mauvaise_structure
- mdp_oublie
- notifications_alertes
- ouvrage_incomplet
- panel_mod0
- permissions
- reputation_utilisateur
- serveur_surchage
- suppression_compte
- upload_contenu_inappropri0
- upload_doublon
- upload_images_mauvaise_qualite

Scenario List and its implementation.

1. Login (**connexion**)

Status: Implemented

Where it is implemented:

1. `login_page()` (user login blueprint)
2. `app.before_request` authentication guard

How it works:

When a user submits login credentials, the system loads registered users from a JSON file and verifies the password using the Argon2 hashing algorithm.

If authentication succeeds, a session is created and the user gains access to protected routes.

If authentication fails, an error message is displayed.

All non-public routes are protected by a global authentication check.

2. User Registration (**inscription**)

Status: Implemented

Where it is implemented:

1. `register_page()`
2. User persistence in `users.json`

How it works:

The system validates required fields (username, email, password).

Passwords are securely hashed before storage.

Duplicate usernames or emails are rejected to prevent account conflicts.

3. Permissions Management (**permissions**)

Status: Implemented (basic)

Where it is implemented:

1. `app.before_request`

How it works:

The application enforces session-based access control.

Unauthenticated users are automatically redirected to the login page.

No role-based permissions (admin vs user) are enforced in the MVP.

4. Duplicate Upload ()

Status: Partially implemented

Where it is implemented:

1. File storage structure under `data/sequestre/scans/`
2. Upload handling logic

How it works:

Books are stored using directory names derived from book identifiers.

If a book with the same name already exists, the filesystem structure implicitly prevents silent duplication.

However, no explicit duplicate warning is currently displayed to the user.

5. Poor Quality Image Upload (`upload_images_mauvaise_qualite`)

Status: Implemented (implicit)

Where it is implemented:

1. Image preprocessing pipeline (`format_small_book.py`)
2. OCR metadata analysis (`ocr.py`)

How it works:

Uploaded scans are preprocessed using cropping, rotation, sharpening, and contrast enhancement.

During OCR extraction, metadata such as `text_quality` and `missing_parts` is generated.

Low-quality scans result in incomplete or unreadable OCR output, which interrupts processing and prompts re-upload.

6. Incomplete Work

Status: Implemented (implicit)

Where it is implemented:

1. OCR extraction ([ocr.py](#))
2. Markdown formatting rules

How it works:

When pages are partially unreadable, missing content is explicitly marked using [\[unreadable\]](#) tokens and metadata fields.

This ensures transparency and prevents silent data corruption.

7. Invalid Structure ()

Status: Implemented

Where it is implemented:

1. Markdown formatting module ([export_md.py](#))

How it works:

Raw OCR text is passed through an LLM-based formatter that enforces strict Markdown structure rules.

Malformed or inconsistent OCR output is normalized into a predictable format.

8. Incorrect Book Identification

Status: Partially implemented

Where it is implemented:

1. Content moderation module ([moderate.py](#))
2. LLM classification logic

How it works:

Book metadata is evaluated by multiple LLMs.

If metadata is ambiguous or inconsistent, the system returns an ["uncertain"](#) classification instead of making unsafe assumptions.

9. Upload of Inappropriate / Copyrighted Content

Status: Implemented

Where it is implemented:

1. `moderate.py`
2. External APIs (LLMs + copyright.gov)
3. Centralized logging system

How it works:

Before storage, book metadata is analyzed using multiple LLMs and a public copyright database.

If a book is determined to be copyrighted, it is rejected and logged, preventing illegal storage.

10. Abusive Behavior

Status: Indirectly handled

Where it is implemented:

1. Authentication system
2. Upload restrictions
3. Logging mechanisms

How it works:

The system limits abusive behavior by requiring authentication, restricting uploads, and logging suspicious or invalid actions.

No explicit abuse scoring system is implemented in the MVP.

11. Admin Management (`gestion_admin`)

Status: Implemented

Where it is implemented:

1. `/dashboard` route
2. Admin dashboard

How it works:

An administrative dashboard allows viewing registered users and system statistics. The interface is read-only in the MVP and does not support user moderation or content deletion.

12. Server Overload (`serveur_surchage`)

Status: Partially implemented

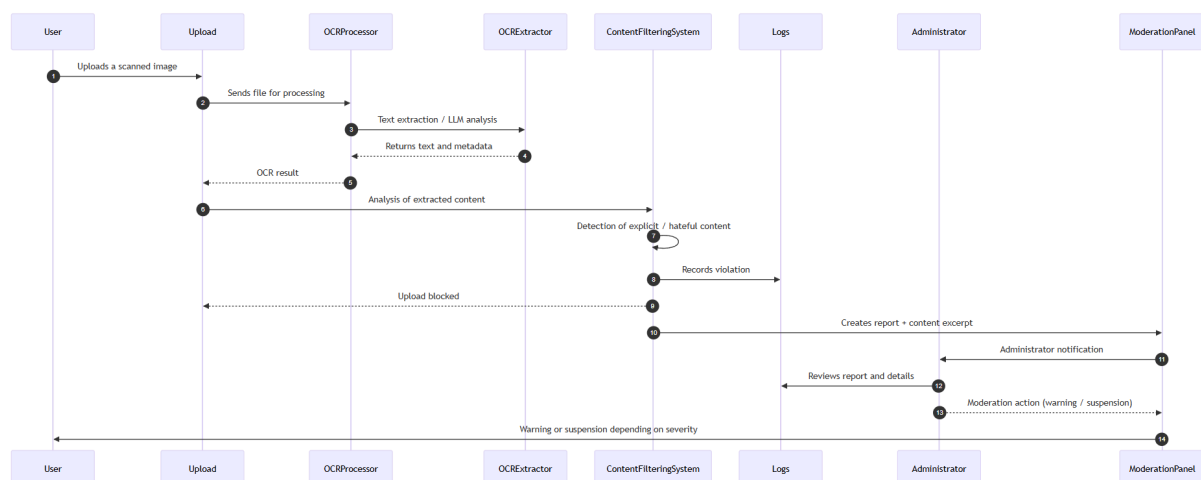
Where it is implemented:

1. API timeouts
2. Retry and cooldown logic in OCR and LLM calls

How it works:

The system mitigates overload effects using request timeouts and retry mechanisms. No load balancing or scaling strategy is implemented.

Use case



Unit tests results

Moderate

These unit tests verify that the `check_book_copyright` function correctly distinguishes between copyrighted works and public-domain books based on their metadata and a simulated consensus from language models. Modern and recent books, such as Harry Potter or A Brief History of Time, are expected to return *True*, indicating that they are likely copyrighted, while older literary works like Les Misérables or Hamlet return *False*, indicating public-domain status. By mocking identical LLM responses for each case, the tests focus on validating that the function interprets the classification results properly and produces the correct boolean outcome.

FS tests

The unit tests for modules that interact directly with the filesystem were intentionally not implemented, as they introduced too many technical complications and potential risks. Testing these components would require creating, modifying, or deleting real files and directories, which could lead to unstable test behavior, environment-dependent results, or accidental data loss. To avoid unsafe side effects and excessive complexity, these filesystem-dependent modules were excluded from unit testing.

Integration tests results

Integration testing was performed manually by executing complete processing pipelines involving multiple system components.

The following integration scenarios were tested:

1. User authentication followed by access to protected routes
2. File upload triggering OCR extraction and Markdown formatting
3. Image preprocessing improving OCR output quality
4. Copyright verification preventing storage of restricted content

All tested pipelines executed successfully, with correct data flow between web routes, CLI tools, LLM services, and filesystem storage.

Failure cases (invalid uploads, OCR errors, copyrighted works) were correctly propagated and handled without system crashes.

These results demonstrate that the system components integrate correctly and fulfill the intended end-to-end behavior.

Validation tests results

Validation testing was conducted to verify that the system meets the client's functional requirements defined in the design brief.

The following requirements were validated:

1. The system accepts scanned documents in PDF and image formats
2. OCR extraction is successfully performed using LLM-based processing
3. Extracted content is transformed into clean, structured Markdown
4. Copyrighted works are detected and rejected before storage

Manual validation confirmed that all core requirements of the MVP (Minimum Viable Product) are fulfilled.

Features identified as out of scope (advanced moderation actions, notifications, reputation system) do not impact the validation of the MVP objectives.

Reversibility

Reversibility was considered as a design principle from the early stages of the project, despite the MVP scope. The objective was to avoid technological lock-in and ensure that the core logic of the system could be adapted or migrated if required.

The backend logic is implemented in **Python**, a high-level and widely adopted language often used as an intermediary ("glue") between systems. Its abstraction level and clear algorithmic structure make the codebase easy to understand, refactor, and translate to another language or framework if needed. As a result, the project's business logic is not tightly coupled to a specific runtime or platform.

The architecture relies on **modular components** (upload handling, OCR processing, filtering, moderation, indexing), which limits interdependencies and facilitates partial replacement. For example, the OCR pipeline is isolated and depends on external APIs and LLMs, which can be replaced by alternative providers or self-hosted solutions without impacting the rest of the system.

The use of **standard and open formats** (Markdown for output, PDFs/images for input) further reinforces reversibility. No proprietary storage format or closed data structure is imposed, making data reuse and migration straightforward.

Additionally, the project deliberately avoids persistent storage of copyrighted works. This design choice not only addresses legal constraints but also simplifies reversibility, as there is no long-term data dependency that would complicate system shutdown, migration, or redesign.

Finally, the absence of heavy infrastructure, databases, or tightly coupled services ensures that the solution remains lightweight. This makes it possible to:

1. Rebuild the system using another framework,
2. Extract and reuse specific modules,
3. Scale or redesign the architecture without rewriting the entire logic.

Overall, the solution favors **conceptual clarity, modularity, and portability**, making it particularly well suited for experimentation, evolution, or reimplementing beyond the MVP phase.